Study Dashboard
Frontend & Backend Structure Reference

A file-by-file map of the React/Vite client and Express/MongoDB server

Architecture at a glance

The frontend is a React single-page application that stores an authentication token locally, calls protected REST APIs through Axios, and provides fallback storage for study sessions. The backend is an Express API that authenticates JWT Bearer requests, persists user-owned records in MongoDB through Mongoose, and exposes focused route modules for each academic feature.

Request flow

Browser UI -> API helper -> Axios token interceptor -> Express route -> JWT middleware -> controller -> Mongoose model -> MongoDB. Responses follow the same path back to the page/component state.

Backend structure

Server-side code is grouped by API responsibility: route modules define endpoints, controllers hold feature logic, and models define the MongoDB data shape.

FILE	ROLE / MAIN FUNCTION
`backend/server.js`	Express entry point: loads environment, connects MongoDB, configures JSON/CORS, mounts current APIs, and starts the server.
`backend/config/db.js`	Connects Mongoose with MONGO_URI and exits on a failed database connection.
`backend/middleware/authMiddleware.js`	Validates Bearer JWTs and attaches the decoded user ID to protected requests.

Backend - models

Each model is user-scoped so academic data remains private to the authenticated account.

FILE	ROLE / MAIN FUNCTION
`models/User.js`	Mongoose account schema: unique username and hashed password.
`models/Schedule.js`	One weekly timetable per user; stores mixed class lists for Sunday through Saturday.
`models/DayRecord.js`	Daily attendance record with totals, holiday flag, per-subject attendance statuses, and a unique user/date index.
`models/Deadline.js`	Task/deadline schema: subject, type, due date, priority, status, completion, subtasks, resources, and estimates.
`models/SemesterEvent.js`	Dated timeline event for a user, with controlled academic event types.
`models/StudySession.js`	Persisted study session: subject, minutes, timer mode, date, completion, and notes.
`models/GpaRecord.js`	One GPA-planning document per user containing target GPA and subject/credit/marks inputs.
`models/ArchiveRecord.js`	One archive document per user with embedded completed-semester history and outcome metrics.
`models/LegacyMonthRecord.js`	Legacy monthly attendance aggregate model (exported as MonthRecord); used by legacy month routes.

Backend - controllers

Controllers implement validation, persistence, aggregation, and feature-level calculations.

FILE	ROLE / MAIN FUNCTION
`controllers/authController.js`	Registers and logs in users with bcrypt password hashes and 7-day JWTs; logout returns a success response.
`controllers/dayController.js`	Upserts daily attendance, derives totals from subject states, returns day/month/overall views, and deletes ranges or all records.
`controllers/scheduleController.js`	Reads a user schedule (with empty-day fallback) and upserts weekly timetable data.
`controllers/semesterController.js`	Lists, creates, and deletes user-owned academic timeline events.
`controllers/studyController.js`	Lists newest-first study sessions, validates/saves sessions, and deletes owned sessions.
`controllers/deadlineController.js`	Deadline CRUD plus analytics, distinct subjects, 7-day workload forecast, insight generation, and rescheduling.
`controllers/monthController.js`	Legacy monthly attendance aggregation: list, increment a week, reset one month, or reset all.

Backend - routes

Routes compose Express endpoints and apply auth middleware to user data APIs.

FILE	ROLE / MAIN FUNCTION
`routes/authRoutes.js`	Public POST endpoints for /register, /login, and /logout; delegates to auth controller.
`routes/scheduleRoutes.js`	Protected GET/PUT schedule endpoints.
`routes/dayRoutes.js`	Protected daily-attendance CRUD and month/overall summary endpoints.
`routes/deadlineRoutes.js`	Protected deadline CRUD, analytics, subjects, forecast, insights, and reschedule endpoints.
`routes/semesterRoutes.js`	Protected GET/POST/DELETE timeline-event endpoints.
`routes/studyRoutes.js`	Protected GET/POST/DELETE study-session endpoints.
`routes/gpaRoutes.js`	Protected GET/PUT GPA setup; seeds a default subject set when absent.
`routes/archiveRoutes.js`	Protected GET/POST/DELETE operations on embedded semester archive items.
`routes/monthRoutes.js`	Legacy protected monthly-attendance API, currently not mounted by server.js.

Backend - project files

Dependency and runtime configuration supporting the server.

FILE	ROLE / MAIN FUNCTION
`backend/package.json`	Backend dependency manifest (Express, Mongoose, JWT, bcrypt, CORS, dotenv, nodemon).
`backend/package-lock.json`	Generated exact dependency lockfile for reproducible backend installs.

Frontend structure

Client-side code is grouped into routed pages, reusable visual components, data-access helpers, shared state, and feature CSS.

FILE	ROLE / MAIN FUNCTION
`main.jsx`	Application bootstrap: React root, BrowserRouter, and Auth/Toast/StudyTimer providers.
`App.jsx`	Top-level routing, login/register gate, scroll reset, logout handling, global mini timer, and mobile navigation.
`context/AuthContext.jsx`	Stores logged-in user/token in localStorage and exposes login/logout state.
`context/ToastContext.jsx`	Global temporary toast notification state and renderer.
`context/StudyTimerContext.jsx`	Global Pomodoro/stopwatch state, ticking, reset controls, and automatic completed-session save.

Frontend - routed pages

Each page represents a primary destination defined in App.jsx.

FILE	ROLE / MAIN FUNCTION
`pages/Hub.jsx`	Home dashboard that loads and caches schedule, attendance, deadlines, forecast, sessions, and timeline events; presents health/activity modules.
`pages/AttendancePage.jsx`	Attendance workspace: month/year selection, schedule display, monthly/overall totals, target threshold, calendar and reset controls.
`pages/DeadlinesPage.jsx`	Deadline hub with list, analytics overview, filters, quick completion/rescheduling actions, timeline tab, and workload panels.
`pages/DeadlineFormPage.jsx`	Create/edit deadline form with metadata, estimated hours, subtasks, and resource links.
`pages/DeadlineDetailPage.jsx`	Single deadline view for task content, linked resources, subtasks, event context, and actions.
`pages/AcademicTimelinePage.jsx`	Timeline event manager with category filtering, search, creation, deletion, and chronological display.
`pages/StudyPage.jsx`	Study area that loads sessions/deadlines, derives available subjects, and composes tracker, stats, and history.
`pages/GpaPage.jsx`	GPA and marks predictor; calculates subject outcomes, required final performance, projected GPA, and persists inputs.
`pages/RoutinePage.jsx`	Weekly class-routine editor with today card, workload overview, timetable matrix, and schedule persistence.
`pages/ArchivePage.jsx`	Completed-semester archive form and growth/history dashboard.
`pages/Login.jsx`	Login form that calls the auth API, updates AuthContext, and links to registration.
`pages/Register.jsx`	Registration form that creates an account, logs in through AuthContext, and links to login.

Frontend - components (attendance, planning, insights)

Reusable UI modules used to assemble the dashboard and feature pages.

FILE	ROLE / MAIN FUNCTION
`components/AttendanceCalendar.jsx`	Month calendar and attendance checklist; loads daily records, saves per-subject states, holidays, and totals.
`components/ScheduleForm.jsx`	Reusable weekday subject-list schedule editor with add/remove/save callbacks.
`components/SummaryCards.jsx`	Presents monthly attendance totals, percentage, and threshold status.
`components/OverallSummary.jsx`	Calculates and displays all-time attendance percentage and target-based status.
`components/ProgressBar.jsx`	Color-coded visual bar for an attendance percentage.
`components/MonthSelector.jsx`	Controlled month dropdown.
`components/ResetSection.jsx`	Confirmation-based controls to clear a selected month or all attendance data.
`components/SemesterTimeline.jsx`	Compact timeline event add/list/delete component.
`components/WorkloadForecast.jsx`	Renders seven-day estimated deadline workload returned by the API.
`components/InsightsPanel.jsx`	Displays actionable deadline insights and offers overdue rescheduling.
`components/AcademicHealthScore.jsx`	Displays weighted health score, component contributions, and low-attendance alert.

Frontend - components (study and navigation)

Timer, history, aggregated activity, and responsive navigation components.

FILE	ROLE / MAIN FUNCTION
`components/StudyTracker.jsx`	Primary study timer supporting stopwatch/custom sessions, breaks, deadline context, and session completion callback.
`components/StudyTimer.jsx`	Alternative focus-timer component with preset modes and session saving flow.
`components/StudyStatsCard.jsx`	Computes totals, today/week activity, and per-subject study breakdowns.
`components/StudyHistoryPanel.jsx`	Filterable study-session history with deletion actions.
`components/UnifiedActivityForecast.jsx`	Combines deadline workload forecast with study-focus pace/streak display.
`components/GlobalMiniTimer.jsx`	Persistent compact control for the timer state supplied by StudyTimerContext.
`components/MobileNav.jsx`	Responsive persistent module navigation with a 'more' sheet.
`components/MobileNotificationDrawer.jsx`	Notification drawer that requests browser permission and renders generated academic alerts.

Frontend - API and utility layer

API modules keep HTTP requests out of UI components; utilities centralize client-side calculations and browser integrations.

FILE	ROLE / MAIN FUNCTION
`api/authApi.js`	Thin Axios wrappers for registration, login, and logout.
`api/attendanceApi.js`	Wrappers for schedule, daily attendance, monthly/overall summary, record list, range deletion, and reset.
`api/deadlinesApi.js`	Wrappers for deadline CRUD, analytics, subjects, workload forecast, insights, and rescheduling.
`api/semesterApi.js`	Wrappers for semester event list/create/delete.
`api/studyApi.js`	Study-session API with localStorage fallback when backend reads/writes/deletes fail.
`api/gpaApi.js`	GET/PUT wrappers for the user GPA setup.
`api/archiveApi.js`	GET/POST/DELETE wrappers for archived semester entries.
`utils/axiosInstance.js`	Configures API base URL and injects the locally stored Bearer token into requests.
`utils/deadlineUtils.js`	Deadline types plus date, urgency, display-label, and subtask-progress helpers.
`utils/healthScore.js`	Weighted academic health score, grade, time formatting, and greeting helpers.
`utils/icsExport.js`	Builds iCalendar text and triggers browser download for events/deadlines.
`utils/mobileNotificationService.js`	Registers service worker, manages notification permission/dismissals, sends native alerts, and generates class/deadline/exam alerts.

Frontend - state, entry point, and styles

Application composition and visual system files.

FILE	ROLE / MAIN FUNCTION
`App.css`	CSS manifest: imports the global and feature stylesheet modules; adds calendar markers.
`styles/global.css`	Shared design tokens, base/reset styles, application layout, headers, and common components.
`styles/Hub.css`	Dashboard-specific layout, background, panels, and responsive styling.
`styles/Attendance.css`	Attendance page, calendar, summaries, and schedule styles.
`styles/Deadlines.css`	Deadline list/detail/form/analytics styling.
`styles/Study.css`	Study timer, history, and study dashboard styling.
`styles/Routine.css`	Routine editor, today's classes, and timetable styling.
`styles/Timeline.css`	Academic timeline cards, filters, and event-list styling.
`styles/Gpa.css`	GPA predictor cards, controls, and responsive layout styling.
`styles/Archive.css`	Semester archive form, metrics, and history styling.
`styles/Auth.css`	Login/register screen layout and form styling.
`styles/MobileNav.css`	Responsive fixed module-navigation styling.
`styles/MobileNotificationDrawer.css`	Notification bell, drawer, alert, and mobile sheet styling.
`styles/GlobalMiniTimer.css`	Floating global timer visual treatment and animation.
`styles/UnifiedActivityForecast.css`	Combined activity/workload forecast card styles.

Project, static, and generated files

Supporting configuration, documentation, assets, and lockfiles. Lockfiles are generated dependency snapshots rather than authored feature code.

FILE	ROLE / MAIN FUNCTION
`frontend/index.html`	Vite HTML shell: document metadata, favicon, root element, and main module entry.
`frontend/vite.config.js`	Vite configuration enabling the React plugin.
`frontend/eslint.config.js`	Flat ESLint config for JavaScript/JSX, React Hooks, and React Refresh.
`frontend/package.json`	Frontend scripts and React/Vite/Axios/router dependencies.
`frontend/package-lock.json`	Generated exact dependency lockfile for reproducible frontend installs.
`frontend/README.md`	Default Vite/React template guidance; not application-specific documentation.
`Readme.md`	Project overview, feature list, stack, deployment notes, and original attendance-tracker workflow.
`public/sw.js`	Service worker lifecycle and notification-click navigation handling.
`public/favicon.svg`	Browser tab icon asset.
`public/icons.svg`	Shared SVG icon sprite/asset.
`public/dashboard-bg.jpg`	Dashboard background image.
`public/section-bg.jpg`	Section background image asset.
`public/bg-teal-mesh.jpg`	Teal mesh background image used by shared layouts.

Notes

This reference inventories the application source and static project files found in the workspace. Dependencies under node_modules and build output are intentionally excluded. The legacy month route/controller/model remain in the repository but server.js does not mount /api/months, while current attendance uses daily records via /api/days.